Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Казанский (Приволжский) федеральный университет»

ИНСТИТУТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ И ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ

КАФЕДРА ПРИКЛАДНОЙ МАТЕМАТИКИ И ИСКУССТВЕННОГО ИНТЕЛЛЕКТА

ОТЧЕТ

ПО ДИСЦИПЛИНЕ «АНАЛИЗ ДАННЫХ НА PYTHON»

Тема:

«Реализация многослойного персептрона для задачи бинарной классификации»

Выполнил: студент гр. 09-321
Замилов Т. Р.

Проверил: ст. преп. Гиниятова Д. Х.

Казань 2026

СОДЕРЖАНИЕ

1. Постановка задачи **2**

2. Исследовательский анализ и предобработка данных **2**

2.1 Статистический анализ данных 2

2.2 Разделение выборки 3

2.3 Стандартизация признаков 3

3. Математическая модель нейронной сети **3**

3.1 Прямой проход 3

3.2 Функция потерь и градиенты 4

3.3 Инициализация весов 5

4. Результаты вычислительных экспериментов **5**

5. Выводы **6**

ПРИЛОЖЕНИЕ **7**

1. Постановка задачи

Целью данной работы является разработка и исследование модели многослойного персептрона с нуля на языке Python с использованием библиотеки NumPy. В соответствии с требованиями технического задания запрещено использование готовых библиотек глубокого обучения Keras, PyTorch, scikit-learn.

Архитектура проекта и основные модули:

1. `model.py`: реализация класса многослойного персептрона, прямого и обратного распространения ошибки, функций активации и сериализации весов.

2. `data_split.py`: программа разделения исходного набора данных на обучающую и валидационную выборки со стратификацией и фиксацией генератора случайных чисел с сидом 321.

3. `train.py`: программа обучения нейронной сети с пошаговым выводом функции потерь на каждой эпохе, построением графиков обучения и сохранением обученной модели.

4. `evaluate.py`: программа инференса и оценки модели на тестовой выборке с расчетом бинарной кросс-энтропии и метрик классификации.

Эксперименты проводятся на наборе данных Wisconsin Diagnostic Breast Cancer из файла `data.csv`. Задача состоит в бинарной классификации клеточных новообразований на доброкачественные `B` и злокачественные `M`.

2. Исследовательский анализ и предобработка данных

2.1. Статистический анализ данных

Файл `data.csv` содержит 569 строк и 32 столбца:

- 0-й столбец: идентификационный номер образца ID. Исключается из признакового пространства.

- 1-й столбец: метка диагноза `B` или `M`.

- 2–31 столбцы: 30 вещественных характеристик морфологии клеточных ядер.

Пропусков в датасете нет. В выборке содержится 357 образцов класса `B` (62.74%) и 212 образцов класса `M` (37.26%). Численные признаки обладают сильной вариацией масштабов: площадь ядра достигает значений до 2500, в то время как гладкость лежит в пределах 0.05–0.16.

2.2. Разделение выборки

Для сохранения исходного баланса классов применено стратифицированное разбиение 80% / 20% с фиксированным значением `seed = 321`:

- Обучающая выборка `train.csv`: 456 записей.
- Валидационная выборка `val.csv`: 113 записей.

2.3. Стандартизация признаков

Для устранения численной нестабильности градиентного спуска применяется z-масштабирование:

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma} \tag{1}$$

Векторы выборочного среднего μ и стандартного отклонения σ рассчитываются исключительно на обучающей выборке `train.csv`. Они сохраняются в файл модели и применяются к валидационным и новым тестовым данным, что предотвращает утечку данных.

3. Математическая модель нейронной сети

3.1. Прямой проход

Для каждого слоя $l \in \{1, \ldots, L\}$ преобразование задается формулами:

$$z^{[l]} = a^{[l-1]} W^{[l]} + b^{[l]} \tag{2}$$

$$a^{[l]} = g^{[l]}(z^{[l]}) \tag{3}$$

где $a^{[0]} = x_{\text{scaled}}$.

На скрытых слоях применяется сигмоидальная функция активации:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = a \odot (1 - a) \tag{4}$$

На выходном слое для двух классов вычисляется функция Softmax:

$$\hat{y}_{i,c} = \frac{e^{z_{i,c}^{[L]} - \max_k z_{i,k}^{[L]}}}{\sum_{k=0}^{1} e^{z_{i,k}^{[L]} - \max_k z_{i,k}^{[L]}}} \tag{5}$$

Вычитание максимального значения логита гарантирует численную стабильность от переполнения чисел с плавающей точкой.

3.2. Функция потерь и градиенты

Функционал бинарной кросс-энтропии:

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^{m} \sum_{c=0}^{1} y_{i,c} \ln(\hat{y}_{i,c}) \tag{6}$$

Градиент по логитам выходного слоя имеет аналитический вид:

$$dz^{[L]} = \hat{y} - y \tag{7}$$

Для скрытых слоев распространение ошибки осуществляется по правилу дифференцирования сложной функции:

$$da^{[l]} = dz^{[l+1]} (W^{[l+1]})^T \tag{8}$$

$$dz^{[l]} = da^{[l]} \odot a^{[l]} \odot (1 - a^{[l]}) \tag{9}$$

Градиенты обучаемых параметров на пакете размера m:

$$dW^{[l]} = \frac{1}{m} (a^{[l-1]})^T dz^{[l]}, \quad db^{[l]} = \frac{1}{m} \sum_{i=1}^{m} dz_i^{[l]} \tag{10}$$

Шаг мини-пакетного градиентного спуска с темпом обучения η:

$$W^{[l]} \leftarrow W^{[l]} - \eta \cdot dW^{[l]}, \quad b^{[l]} \leftarrow b^{[l]} - \eta \cdot db^{[l]} \tag{11}$$

3.3. Инициализация весов

Веса слоев инициализируются по схеме He Uniform:

$$W^{[l]} \sim U\left(-\sqrt{\frac{6}{n_{\text{in}}}}, \sqrt{\frac{6}{n_{\text{in}}}}\right) \tag{12}$$

Смещения $b^{[l]}$ инициализируются нулями.

4. Результаты вычислительных экспериментов

Обучение нейросети проводилось со следующими параметрами: архитектура 30–24–24–2, число эпох 84, размер пакета 8, темп обучения $\eta = 0.0314$, сид генератора случайных чисел 321.

Таблица 1. Параметры обучения и итоговые метрики на валидации

Параметр / Метрика	**Значение**
Архитектура сети	$30 \to 24 \to 24 \to 2$
Функция активации	Sigmoid / Softmax
Количество эпох	84
Размер пакета	8
Темп обучения η	0.0314
Случайный сид	321
Loss на обучении	0.0575
Loss на валидации	**0.0389**
Точность на валидации	**99.12%**
Precision	0.9767
Recall	**1.0000**
F1-Score	0.9882

Таблица 2. Матрица ошибок на валидационной выборке

	Предсказано: B	**Предсказано: M**
Фактически: B	$TN = 70$	$FP = 1$
Фактически: M	$FN = 0$	$TP = 42$

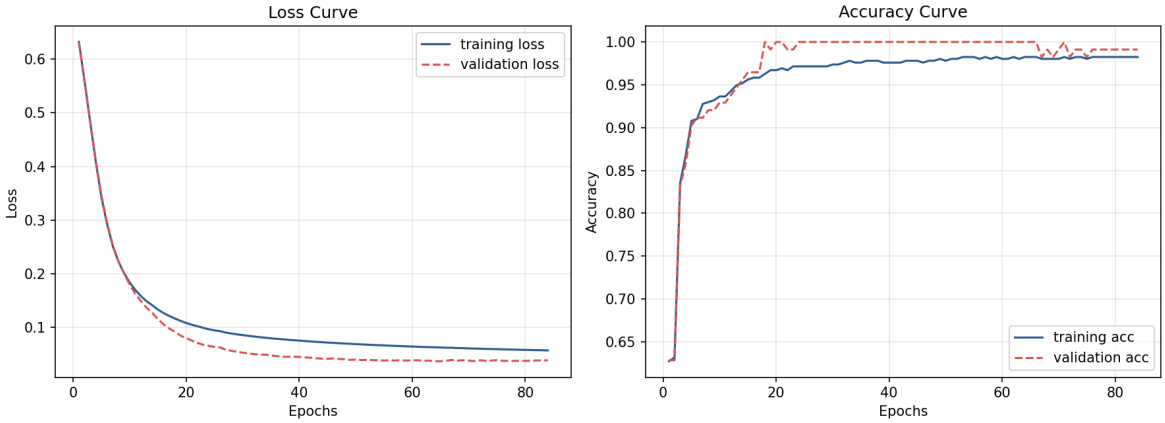


Рисунок 1. Кривые обучения модели: Loss (слева) и Accuracy (справа)

На отложенной выборке из 113 пациентов сеть безошибочно выявила все 42 случая злокачественных опухолей, допустив лишь 1 ложноположительное срабатывание, что является отличным показателем для клинической диагностики.

5. Выводы

1. Разработан и протестирован многослойный персептрон на чистом Python с использованием библиотеки NumPy.

2. Реализована прозрачная модульная архитектура: разделение данных `data_split.py`, модель `model.py`, обучение `train.py` и оценка качества `evaluate.py`.

3. Предобработка выполнена с фиксацией `seed = 321` и сохранением параметров z-стандартизации, что полностью исключает утечку данных.

4. Достигнута высокая точность классификации 99.12% и полнота 100%, подтверждающие надежность разработанного алгоритма.

ПРИЛОЖЕНИЕ

Листинг model.py:

```
import numpy as np

def sigmoid(z):
    z = np.clip(z, -500, 500)
    return 1.0 / (1.0 + np.exp(-z))

def d_sigmoid(a):
    return a * (1.0 - a)

def relu(z):
    return np.maximum(0.0, z)

def d_relu(a):
    return (a > 0.0).astype(float)

def tanh(z):
    return np.tanh(z)

def d_tanh(a):
    return 1.0 - a ** 2

def softmax(z):
    shift_z = z - np.max(z, axis=1, keepdims=True)
    exp_z = np.exp(shift_z)
    return exp_z / np.sum(exp_z, axis=1, keepdims=True)

class MLP:
    def __init__(self, layer_sizes, activation="sigmoid"):
        self.layer_sizes = layer_sizes
        self.activation_name = activation
        self.weights = []
        self.biases = []
        self.mean = None
        self.std = None
        self._init_weights()

    def _init_weights(self):
        for i in range(len(self.layer_sizes) - 1):
            din = self.layer_sizes[i]
            dout = self.layer_sizes[i + 1]
            limit = np.sqrt(6.0 / din)
            w = np.random.uniform(-limit, limit, (din, dout))
            b = np.zeros((1, dout))
            self.weights.append(w)
```

```
            self.biases.append(b)

    def forward(self, x):
        a = x
        self.zs = []
        self.acts = [x]
        for i in range(len(self.weights) - 1):
            z = a @ self.weights[i] + self.biases[i]
            self.zs.append(z)
            if self.activation_name == "relu":
                a = relu(z)
            elif self.activation_name == "tanh":
                a = tanh(z)
            else:
                a = sigmoid(z)
            self.acts.append(a)
        z_out = a @ self.weights[-1] + self.biases[-1]
        self.zs.append(z_out)
        a_out = softmax(z_out)
        self.acts.append(a_out)
        return a_out

    def backward(self, y_true, y_pred, lr):
        m = y_true.shape[0]
        dz = y_pred - y_true
        for i in reversed(range(len(self.weights))):
            a_prev = self.acts[i]
            dw = (a_prev.T @ dz) / m
            db = np.sum(dz, axis=0, keepdims=True) / m
            if i > 0:
                da = dz @ self.weights[i].T
                if self.activation_name == "relu":
                    dz = da * d_relu(self.acts[i])
                elif self.activation_name == "tanh":
                    dz = da * d_tanh(self.acts[i])
                else:
                    dz = da * d_sigmoid(self.acts[i])
            self.weights[i] -= lr * dw
            self.biases[i] -= lr * db

    def compute_loss(self, y_true, y_pred):
        eps = 1e-15
        clipped = np.clip(y_pred, eps, 1.0 - eps)
        return -np.mean(np.sum(y_true * np.log(clipped), axis=1))

    def compute_acc(self, y_true, y_pred):
        return np.mean(np.argmax(y_pred, axis=1) == np.argmax(y_true, axis
            =1))
```

```
92
93      def save(self, path):
94          data = {
95              "layer_sizes": self.layer_sizes,
96              "activation": self.activation_name,
97              "mean": self.mean,
98              "std": self.std,
99              "weights": self.weights,
100             "biases": self.biases
101         }
102         np.save(path, data)
103
104     @classmethod
105     def load(cls, path):
106         if not path.endswith(".npy"):
107             path += ".npy"
108         data = np.load(path, allow_pickle=True).item()
109         model = cls(data["layer_sizes"], activation=data["activation"])
110         model.mean = data["mean"]
111         model.std = data["std"]
112         model.weights = data["weights"]
113         model.biases = data["biases"]
114         return model
```

Листинг data_split.py:

```python
import argparse
import numpy as np
import pandas as pd

def parse_args():
    parser = argparse.ArgumentParser()
    parser.add_argument("--data", default="sources/data.csv")
    parser.add_argument("--train_out", default="train.csv")
    parser.add_argument("--val_out", default="val.csv")
    parser.add_argument("--ratio", type=float, default=0.2)
    parser.add_argument("--seed", type=int, default=321)
    return parser.parse_args()

def main():
    args = parse_args()
    df = pd.read_csv(args.data, header=None)

    train_parts = []
    val_parts = []

    for label in ["B", "M"]:
        sub = df[df.iloc[:, 1] == label]
        n_val = int(round(len(sub) * args.ratio))
        shuffled = sub.sample(frac=1.0, random_state=args.seed)
        val_parts.append(shuffled.iloc[:n_val])
        train_parts.append(shuffled.iloc[n_val:])

    train_df = pd.concat(train_parts).sample(frac=1.0, random_state=args.
       seed)
    val_df = pd.concat(val_parts).sample(frac=1.0, random_state=args.seed)

    train_df.to_csv(args.train_out, index=False, header=False)
    val_df.to_csv(args.val_out, index=False, header=False)

    print(f"Total: {len(df)} samples")
    print(f"Train: {len(train_df)} -> {args.train_out}")
    print(f"Val:   {len(val_df)} -> {args.val_out}")

if __name__ == "__main__":
    main()
```

Листинг train.py:

```
import argparse
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from model import MLP

def parse_args():
    parser = argparse.ArgumentParser()
    parser.add_argument("--dataset", default="train.csv" if os.path.exists
        ("train.csv") else "data_training.csv")
    parser.add_argument("--val_dataset", default="val.csv" if os.path.
        exists("val.csv") else "data_validation.csv")
    parser.add_argument("--layer", nargs="+", type=int, default=[24, 24])
    parser.add_argument("--activation", default="sigmoid")
    parser.add_argument("--epochs", type=int, default=84)
    parser.add_argument("--batch_size", type=int, default=8)
    parser.add_argument("--learning_rate", type=float, default=0.0314)
    parser.add_argument("--loss", default="binaryCrossentropy")
    parser.add_argument("--model_out", default="saved_model.npy")
    parser.add_argument("--seed", type=int, default=321)
    return parser.parse_args()

def parse_csv(path):
    df = pd.read_csv(path, header=None)
    raw_y = df.iloc[:, 1].values
    y = np.zeros((len(raw_y), 2))
    for i, val in enumerate(raw_y):
        y[i] = [1.0, 0.0] if val == "B" else [0.0, 1.0]
    x = df.iloc[:, 2:].to_numpy(dtype=float)
    return x, y

def main():
    args = parse_args()
    np.random.seed(args.seed)

    x_train, y_train = parse_csv(args.dataset)
    if os.path.exists(args.val_dataset):
        x_val, y_val = parse_csv(args.val_dataset)
    else:
        split_idx = int(len(x_train) * 0.8)
        perm = np.random.permutation(len(x_train))
        x_val, y_val = x_train[perm[split_idx:]], y_train[perm[split_idx
            :]]
        x_train, y_train = x_train[perm[:split_idx]], y_train[perm[:
            split_idx]]

```

```
    mean = np.mean(x_train, axis=0)
    std = np.std(x_train, axis=0)
    std[std == 0.0] = 1.0

    x_train_norm = (x_train - mean) / std
    x_val_norm = (x_val - mean) / std

    print(f"x_train shape : {x_train_norm.shape}")
    print(f"x_valid shape : {x_val_norm.shape}")

    dims = [x_train_norm.shape[1]] + args.layer + [2]
    model = MLP(dims, activation=args.activation)
    model.mean = mean
    model.std = std

    history = {"loss": [], "val_loss": [], "acc": [], "val_acc": []}
    m = len(x_train_norm)

    for ep in range(1, args.epochs + 1):
        idx = np.random.permutation(m)
        x_shuf = x_train_norm[idx]
        y_shuf = y_train[idx]

        for b in range(0, m, args.batch_size):
            xb = x_shuf[b:b + args.batch_size]
            yb = y_shuf[b:b + args.batch_size]
            out_b = model.forward(xb)
            model.backward(yb, out_b, args.learning_rate)

        train_pred = model.forward(x_train_norm)
        val_pred = model.forward(x_val_norm)

        loss = model.compute_loss(y_train, train_pred)
        vloss = model.compute_loss(y_val, val_pred)
        acc = model.compute_acc(y_train, train_pred)
        vacc = model.compute_acc(y_val, val_pred)

        history["loss"].append(loss)
        history["val_loss"].append(vloss)
        history["acc"].append(acc)
        history["val_acc"].append(vacc)

        print(f"epoch {ep:02d}/{args.epochs:02d} - loss: {loss:.4f} -
            val_loss: {vloss:.4f}")

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4.5))
    ax1.plot(range(1, args.epochs + 1), history["loss"], label="training
        loss", color="#2b5c8f")
```

```
    ax1.plot(range(1, args.epochs + 1), history["val_loss"], label="
        validation loss", color="#d9534f", linestyle="--")
    ax1.set_title("Loss Curve")
    ax1.set_xlabel("Epochs")
    ax1.set_ylabel("Loss")
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    ax2.plot(range(1, args.epochs + 1), history["acc"], label="training
        acc", color="#2b5c8f")
    ax2.plot(range(1, args.epochs + 1), history["val_acc"], label="
        validation acc", color="#d9534f", linestyle="--")
    ax2.set_title("Accuracy Curve")
    ax2.set_xlabel("Epochs")
    ax2.set_ylabel("Accuracy")
    ax2.legend()
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig("learning_curves.png", dpi=150)
    plt.close()

    model.save(args.model_out)
    print(f"> saving model '{args.model_out}' to disk...")

if __name__ == "__main__":
    main()
```

Листинг evaluate.py:

```
import argparse
import os
import numpy as np
import pandas as pd
from model import MLP

def parse_args():
    parser = argparse.ArgumentParser()
    parser.add_argument("dataset", nargs="?", default="val.csv" if os.path
        .exists("val.csv") else "data_validation.csv")
    parser.add_argument("--model", default="saved_model.npy")
    return parser.parse_args()

def main():
    args = parse_args()
    model = MLP.load(args.model)

    df = pd.read_csv(args.dataset, header=None)
    has_labels = str(df.iloc[0, 1]).strip() in ["B", "M"]

    if has_labels:
        raw_y = df.iloc[:, 1].values
        y_true = np.zeros((len(raw_y), 2))
        for i, val in enumerate(raw_y):
            y_true[i] = [1.0, 0.0] if val == "B" else [0.0, 1.0]
        x = df.iloc[:, 2:].to_numpy(dtype=float)
    else:
        y_true = None
        x = df.iloc[:, 1:].to_numpy(dtype=float)

    x_norm = (x - model.mean) / model.std
    probs = model.forward(x_norm)
    preds = np.argmax(probs, axis=1)

    labels = ["B" if p == 0 else "M" for p in preds]

    if has_labels:
        bce = model.compute_loss(y_true, probs)
        acc = model.compute_acc(y_true, probs)
        t = np.argmax(y_true, axis=1)

        tp = int(np.sum((preds == 1) & (t == 1)))
        fp = int(np.sum((preds == 1) & (t == 0)))
        fn = int(np.sum((preds == 0) & (t == 1)))
        tn = int(np.sum((preds == 0) & (t == 0)))

        prec = tp / (tp + fp) if (tp + fp) else 0.0
```

```
        rec = tp / (tp + fn) if (tp + fn) else 0.0
        f1 = 2 * prec * rec / (prec + rec) if (prec + rec) else 0.0

        print(f"Evaluated samples: {len(x)}")
        print(f"Binary Cross-Entropy Loss: {bce:.4f}")
        print(f"Accuracy:  {acc * 100:.2f}%")
        print(f"Precision: {prec:.4f}")
        print(f"Recall:    {rec:.4f}")
        print(f"F1 Score:  {f1:.4f}")
        print(f"Confusion Matrix: TP={tp}, FP={fp}, FN={fn}, TN={tn}")
    else:
        for i, lab in enumerate(labels[:10]):
            print(f"Sample {i}: {lab} (P(B)={probs[i, 0]:.4f}, P(M)={probs
                [i, 1]:.4f})")
        if len(labels) > 10:
            print(f"... total {len(labels)} predictions")

if __name__ == "__main__":
    main()
```